Prototype: CNN + Grad-CAM on CBIS-DDSM subset

Import Libraries

```
# %%
# Prototype: CNN + Grad-CAM on CBIS-DDSM subset

import os
import sys
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

import torch
import torch.nn as nn
from torch.utils.data import DataLoader
from torchvision import models

# Make project root importable
ROOT_DIR = os.path.abspath(os.path.join(".."))
if ROOT_DIR not in sys.path:
    sys.path.append(ROOT_DIR)

from utils.dataset import CBISDicomDataset  # you'll create/adjust this
from utils.transforms import get_preprocessing_transforms
from utils.gradcam import GradCAM

device = "cuda" if torch.cuda.is_available() else "cpu"
device
```

```
'cpu'
```

Paths

```
RAW_ROOT = os.path.join(ROOT_DIR, "data", "raw", "CBIS-DDSM-All-doiJNLP-zzWs5zfZ", "cbis_dds
METADATA_MASS_TRAIN = os.path.join(ROOT_DIR, "data", "raw", "mass_case_description_train_set
METADATA_MASS_TEST = os.path.join(ROOT_DIR, "data", "raw", "mass_case_description_test_set.c

print(RAW_ROOT)
print(METADATA_MASS_TRAIN)
```

```
/Users/gibionmakiwa/MyDocuments/610MtSundanceBayW/University_of_London_Computer_Science_BSc/
/Users/gibionmakiwa/MyDocuments/610MtSundanceBayW/University_of_London_Computer_Science_BSc/
```

Load metadata and build a small prototype subset (e.g., 200 cases)

```
CSV_PATH = os.path.join(
    ROOT_DIR,
    "data",
    "raw",
    "CBIS-DDSM-All-doiJNLP-zzWs5zfZ",
    "metadata",
    "mass_train_with_dicom_paths.csv"
)

meta = pd.read_csv(CSV_PATH)

# Keep only rows that have a valid DICOM path
meta = meta[meta["actual_dicom_path"].notna()].reset_index(drop=True)

# Sample 200 for prototype
meta_subset = meta.sample(n=200, random_state=42).reset_index(drop=True)

# Ensure label is binary
meta_subset["label"] = (meta_subset["pathology"] == "MALIGNANT").astype(int)

meta_subset.head()
```

```
  patient_id  breast_density left or right breast image view  abnormality id  \
0    P_00976               1                 LEFT        MLO               1
1    P_01394               2                RIGHT        MLO               1
2    P_00871               2                 LEFT         CC               1
3    P_00068               3                RIGHT         CC               1
4    P_01849               1                RIGHT        MLO               1

  abnormality type  mass shape mass margins  assessment  \
0             mass   IRREGULAR     OBSCURED           0
1             mass   IRREGULAR   SPICULATED           5
2             mass   IRREGULAR  ILL_DEFINED           4
3             mass   IRREGULAR   SPICULATED           4
4             mass  LYMPH_NODE          NaN           2

                pathology  subtlety  \
0                  BENIGN         2
1               MALIGNANT         5
2               MALIGNANT         3
3               MALIGNANT         4
4  BENIGN_WITHOUT_CALLBACK         3
```

```
                                    image file path  \
0  Mass-Training_P_00976_LEFT_MLO/1.3.6.1.4.1.959...
1  Mass-Training_P_01394_RIGHT_MLO/1.3.6.1.4.1.95...
2  Mass-Training_P_00871_LEFT_CC/1.3.6.1.4.1.9590...
3  Mass-Training_P_00068_RIGHT_CC/1.3.6.1.4.1.959...
4  Mass-Training_P_01849_RIGHT_MLO/1.3.6.1.4.1.95...

                              cropped image file path  \
0  Mass-Training_P_00976_LEFT_MLO_1/1.3.6.1.4.1.9...
1  Mass-Training_P_01394_RIGHT_MLO_1/1.3.6.1.4.1....
2  Mass-Training_P_00871_LEFT_CC_1/1.3.6.1.4.1.95...
3  Mass-Training_P_00068_RIGHT_CC_1/1.3.6.1.4.1.9...
4  Mass-Training_P_01849_RIGHT_MLO_1/1.3.6.1.4.1....

                                   ROI mask file path  \
0  Mass-Training_P_00976_LEFT_MLO_1/1.3.6.1.4.1.9...
1  Mass-Training_P_01394_RIGHT_MLO_1/1.3.6.1.4.1....
2  Mass-Training_P_00871_LEFT_CC_1/1.3.6.1.4.1.95...
3  Mass-Training_P_00068_RIGHT_CC_1/1.3.6.1.4.1.9...
4  Mass-Training_P_01849_RIGHT_MLO_1/1.3.6.1.4.1....

                         tcia_folder  \
0   Mass-Training_P_00976_LEFT_MLO_1
1  Mass-Training_P_01394_RIGHT_MLO_1
2      Mass-Training_P_00871_LEFT_CC
3   Mass-Training_P_00068_RIGHT_CC_1
4    Mass-Training_P_01849_RIGHT_MLO

                                  actual_dicom_path  label
0  /Users/gibionmakiwa/MyDocuments/610MtSundanceB...      0
1  /Users/gibionmakiwa/MyDocuments/610MtSundanceB...      1
2  /Users/gibionmakiwa/MyDocuments/610MtSundanceB...      1
3  /Users/gibionmakiwa/MyDocuments/610MtSundanceB...      1
4  /Users/gibionmakiwa/MyDocuments/610MtSundanceB...      0
```

Inspect how DICOM paths are encoded in metadata

```
# %%
# Inspect how DICOM paths are encoded in metadata

meta_subset[["patient_id", "image file path", "label"]].head()
```

```
  patient_id                                   image file path  label
0    P_00976  Mass-Training_P_00976_LEFT_MLO/1.3.6.1.4.1.959...      0
1    P_01394  Mass-Training_P_01394_RIGHT_MLO/1.3.6.1.4.1.95...      1
2    P_00871  Mass-Training_P_00871_LEFT_CC/1.3.6.1.4.1.9590...      1
```

```
3    P_00068  Mass-Training_P_00068_RIGHT_CC/1.3.6.1.4.1.959...      1
4    P_01849  Mass-Training_P_01849_RIGHT_MLO/1.3.6.1.4.1.95...      0
```

Create PyTorch Dataset using utils/dataset.py

```
# %%
# Create PyTorch Dataset using your utils/dataset.py

from torch.utils.data import random_split

transform = get_preprocessing_transforms()  # from utils/transforms.py

full_dataset = CBISDicomDataset(
    metadata=meta_subset,
    load_masks=True
)

len(full_dataset)
```

```
<utils.dataset.CBISDicomDataset object at 0x12936acd0>
```

Split into train/val for the prototype

```
# %%
# Split into train/val for the prototype

train_size = int(0.8 * len(full_dataset))
val_size = len(full_dataset) - train_size
train_dataset, val_dataset = random_split(full_dataset, [train_size, val_size])

train_loader = DataLoader(train_dataset, batch_size=16, shuffle=True, num_workers=2)
val_loader = DataLoader(val_dataset, batch_size=16, shuffle=False, num_workers=2)

len(train_dataset), len(val_dataset)
```

```
(160, 40)
```

Visual sanity check: show one sample

```
# %%
# Visual sanity check: show one sample

img, mask, label = val_dataset[0]

input_tensor = img.unsqueeze(0).to(device)
cam = gradcam(input_tensor)
```

```
img_np = img.permute(1,2,0).numpy()
mask_np = mask.squeeze().numpy()

plt.figure(figsize=(12,4))

plt.subplot(1,3,1)
plt.imshow(img_np, cmap="gray")
plt.title("Original")

plt.subplot(1,3,2)
plt.imshow(mask_np, cmap="gray")
plt.title("ROI Mask")

plt.subplot(1,3,3)
plt.imshow(img_np, cmap="gray")
plt.imshow(cam, cmap="jet", alpha=0.4)
plt.title("Grad-CAM Overlay")

plt.show()
```

```
---------------------------------------------------------------------------
error                                     Traceback (most recent call last)
Cell In[7], line 4
      1 # %%
      2 # Visual sanity check: show one sample
----> 4 img, label = full_dataset[0]
      5 img_np = np.transpose(img.numpy(), (1, 2, 0))
      6 plt.imshow(img_np, cmap="gray")

File ~/MyDocuments/610MtSundanceBayW/University_of_London_Computer_Science_BSc/CM3070 Comput
     26 img = np.stack([img, img, img], axis=-1)
     28 if self.transform:
---> 29     img = self.transform(img)
     31 return img, label

File /usr/local/lib/python3.8/site-packages/torchvision/transforms/transforms.py:95, in Comp
     93 def __call__(self, img):
     94     for t in self.transforms:
---> 95         img = t(img)
     96     return img

File ~/MyDocuments/610MtSundanceBayW/University_of_London_Computer_Science_BSc/CM3070 Comput
      8 if img.ndim == 2:
      9     img = cv2.cvtColor(img, cv2.COLOR_GRAY2RGB)
---> 11 img = cv2.equalizeHist(img[:, :, 0])
     12 img = cv2.cvtColor(img, cv2.COLOR_GRAY2RGB)
```

```
     13 img = cv2.resize(img, (224, 224))

error: OpenCV(4.12.0) /Users/runner/work/opencv-python/opencv-python/opencv/modules/imgproc/
```

Build and configure ResNet50

```
# %%
# Build and configure ResNet50

resnet = models.resnet50(weights=models.ResNet50_Weights.IMAGENET1K_V2)
num_features = resnet.fc.in_features
resnet.fc = nn.Linear(num_features, 1)

resnet = resnet.to(device)

criterion = nn.BCEWithLogitsLoss()
optimizer = torch.optim.Adam(resnet.parameters(), lr=1e-4)
```

Training loop (prototype: ~5–10 epochs)

```
# %%
# Training loop (prototype: ~5-10 epochs)

from tqdm.auto import tqdm

def train_one_epoch(model, loader, optimizer, criterion, device):
    model.train()
    running_loss = 0.0

    for imgs, labels in tqdm(loader, desc="Train", leave=False):
        imgs = imgs.to(device)
        labels = labels.float().to(device)

        optimizer.zero_grad()
        outputs = model(imgs).squeeze(1)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        running_loss += loss.item() * imgs.size(0)

    return running_loss / len(loader.dataset)


def evaluate(model, loader, criterion, device):
```

```
    model.eval()
    running_loss = 0.0
    all_outputs = []
    all_labels = []

    with torch.no_grad():
        for imgs, labels in tqdm(loader, desc="Val", leave=False):
            imgs = imgs.to(device)
            labels = labels.float().to(device)

            outputs = model(imgs).squeeze(1)
            loss = criterion(outputs, labels)

            running_loss += loss.item() * imgs.size(0)
            all_outputs.append(outputs.cpu())
            all_labels.append(labels.cpu())

    all_outputs = torch.cat(all_outputs)
    all_labels = torch.cat(all_labels)

    probs = torch.sigmoid(all_outputs)
    preds = (probs > 0.5).int()
    acc = (preds == all_labels.int()).float().mean().item()

    return running_loss / len(loader.dataset), acc


EPOCHS = 10
for epoch in range(EPOCHS):
    train_loss = train_one_epoch(resnet, train_loader, optimizer, criterion, device)
    val_loss, val_acc = evaluate(resnet, val_loader, criterion, device)
    print(f"Epoch {epoch+1}/{EPOCHS} | Train Loss: {train_loss:.4f} | Val Loss: {val_loss:.4
```

Save prototype model

```
## # %%
# Save prototype model

os.makedirs(os.path.join(ROOT_DIR, "models"), exist_ok=True)
MODEL_PATH = os.path.join(ROOT_DIR, "models", "resnet50_prototype.pth")
torch.save(resnet.state_dict(), MODEL_PATH)
MODEL_PATH
```

Grad-CAM: use utils/gradcam.py

```
# %%
# Grad-CAM: use your utils/gradcam.py

# We assume utils/gradcam.py defines a GradCAM class similar to:
# GradCAM(model, target_layer)

target_layer = resnet.layer4[-1]
gradcam = GradCAM(resnet, target_layer, device=device)
```

Pick a validation sample and generate Grad-CAM

```
# %%
# Pick a validation sample and generate Grad-CAM

resnet.eval()

img, label = val_dataset[0]
input_tensor = img.unsqueeze(0).to(device)

cam = gradcam(input_tensor)  # expected shape: (H, W) normalised 0-1

# Convert original image back to HxW or HxWx3 for overlay
img_np = np.transpose(img.numpy(), (1, 2, 0))

heatmap = plt.cm.jet(cam)[..., :3]  # RGB
overlay = 0.4 * heatmap + 0.6 * img_np

plt.figure(figsize=(12, 4))
plt.subplot(1, 3, 1)
plt.imshow(img_np, cmap="gray")
plt.title(f"Original (label={int(label)})")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(cam, cmap="jet")
plt.title("Grad-CAM")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(overlay)
plt.title("Overlay")
plt.axis("off")

plt.tight_layout()
```

```
plt.show()
```

Loop over several samples and save Grad-CAM overlays to results/heatmaps

```
# %%
# (Optional) Loop over several samples and save Grad-CAM overlays to results/heatmaps

HEATMAP_DIR = os.path.join(ROOT_DIR, "results", "heatmaps")
os.makedirs(HEATMAP_DIR, exist_ok=True)

for i in range(10):
    img, label = val_dataset[i]
    input_tensor = img.unsqueeze(0).to(device)
    cam = gradcam(input_tensor)

    img_np = np.transpose(img.numpy(), (1, 2, 0))
    heatmap = plt.cm.jet(cam)[..., :3]
    overlay = 0.4 * heatmap + 0.6 * img_np

    plt.imsave(os.path.join(HEATMAP_DIR, f"sample_{i}_overlay_label_{int(label)}.png"), over
```